Nikolski Operators Functions Systems Benchmark

Natural language statement	Domain	GroundTruth
1.3.2. Theorem (A. Beurling, H. Helson). Let E subset L2, zE proper_subset E. Then there exists a measurable function Theta, unique up to a multiplicative constant of modulus 1, such that |Theta| = 1 a.e. on T and E = ThetaH2.	Hardy spaces	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1}) : C := limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1) noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp> def HardyClass (p : R>=0inf) (f : C -> C) : Prop := DifferentiableOn C f (Metric.ball (0 : C) 1) /\ exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 -> eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C def HasTaylorSeries (f : C -> C) (a : N -> C) : Prop := forall z in Metric.ball (0 : C) 1, HasSum (fun n : N => a n * z ^ n) (f z) def HardySquareSummable (a : N -> C) : Prop := Summable fun n : N => ||a n|| ^ 2 noncomputable def CauchyProduct (a b : N -> C) : N -> C := fun n : N => Finset.sum (Finset.range (n + 1)) (fun k => a k * b (n - k)) def IsComplexLinearSubspace (M : Set (N -> C)) : Prop := (0 : N -> C) in M /\ forall a b : C, forall f g : N -> C, f in M -> g in M -> a * f + b * g in M def ShiftInvariant (M : Set (N -> C)) : Prop := forall f in M, (fun n : N => if n = 0 then 0 else f (n - 1)) in M def InnerFunction (theta : C -> C) : Prop := HardyClass top theta /\ forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), ||boundaryValue theta (unitCirclePoint t)|| = 1 def InnerGeneratedSubspace (M : Set (N -> C)) (htheta : N -> C) : Prop := HardySquareSummable htheta /\ M = {f : N -> C | exists g : N -> C, HardySquareSummable g /\ f = CauchyProduct htheta g} theorem nikolski_A_1_3_beurling_invariant_subspaces {M : Set (N -> C)} (hlinear : IsComplexLinearSubspace M) (hclosed : IsClosed M) (hshift : ShiftInvariant M) (hne : exists f in M, f != 0) : exists theta : C -> C, exists htheta : N -> C, InnerFunction theta /\ HasTaylorSeries theta htheta /\ InnerGeneratedSubspace M htheta /\ forall eta : C -> C, forall heta : N -> C, InnerFunction eta -> HasTaylorSeries eta heta -> InnerGeneratedSubspace M heta -> exists c : C, ||c|| = 1 /\ forall z in Metric.ball (0 : C) 1, eta z = c * theta z := by sorry
2.4.1. Theorem (V. Smirnov, 1928). Let f in H2, f != 0. Then there exist an inner function finn in H2 and an outer function fout in H2 such that f = finnfout. Moreover, such a factorization is unique up to a constant factor, and E_f = finnH2. Recall that a function f in H2 is called inner if |f| = 1 a.e. on T. It is called outer if E_f = H2.	Hardy spaces	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1}) : C := limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1) noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp> def HardyClass (p : R>=0inf) (f : C -> C) : Prop := DifferentiableOn C f (Metric.ball (0 : C) 1) /\ exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 -> eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C def InnerFunction (theta : C -> C) : Prop := HardyClass top theta /\ forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), ||boundaryValue theta (unitCirclePoint t)|| = 1

Natural language statement	Domain	GroundTruth

		def OuterFunction (p : R>=0inf) (g : C -> C) : Prop :=
		  HardyClass p g /\ exists c : C, ||c|| = 1 /\ forall z in Metric.ball (0 :
		  C) 1,
		  g z = c * Complex.exp
		  ((2 * Real.pi : C)^-1 *
		  integral t in Set.Ioc 0 (2 * Real.pi),
		  ((unitCirclePoint t).1 + z) / ((unitCirclePoint t).1 - z) *
		  Real.log ||boundaryValue g (unitCirclePoint t)||)
		
		theorem nikolski_A_2_4_inner_outer_factorization
		  {f : C -> C} (hf : HardyClass 2 f) (hnonzero : exists z, f z != 0) :
		  exists theta g : C -> C, InnerFunction theta /\ OuterFunction 2 g /\
		  (forall z in Metric.ball (0 : C) 1, f z = theta z * g z) /\
		  forall theta' g' : C -> C, InnerFunction theta' -> OuterFunction 2 g'
		  ->
		  (forall z in Metric.ball (0 : C) 1, f z = theta' z * g' z) ->
		  exists c : C, ||c|| = 1 /\ forall z in Metric.ball (0 : C) 1,
		  theta' z = c * theta z /\ g' z = star c * g z := by
		  sorry

3.6.1. Corollary. If g in H1, g != 0, then log |g| in L1(T).
In particular, if g in H1 and m{t in T : g(t) = 0} > 0, then g
= 0.
	Hardy spaces	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1})
		: C :=
		  limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1)
		
		noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} :=
		  <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp>
		
		def HardyClass (p : R>=0inf) (f : C -> C) : Prop :=
		  DifferentiableOn C f (Metric.ball (0 : C) 1) /\
		  exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 ->
		  eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p
		  (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C
		
		theorem nikolski_A_3_6_boundary_uniqueness
		  {p : R>=0inf} {f : C -> C} (hp : p != 0) (hf : HardyClass p f) :
		  ((exists z in Metric.ball (0 : C) 1, f z != 0) ->
		  IntegrableOn (fun t : R =>
		  Real.log ||boundaryValue f (unitCirclePoint t)||) (Set.Ioc 0 (2 *
		  Real.pi))) /\
		  forall E : Set {z : C // ||z|| = 1},
		  0 < volume {t in Set.Ioc 0 (2 * Real.pi) | unitCirclePoint t in E} ->
		  (forall zeta in E, boundaryValue f zeta = 0) -> forall z in
		  Metric.ball (0 : C) 1, f z = 0 := by
		  sorry

3.7.1. Lemma (Blaschke condition, interior uniqueness
theorem). Suppose f in Hol(D), f != 0, and let (lambdan)n>=1
be the zero sequence of f in D, each zero is repeated
according to its multiplicity. Suppose that limr->1 integralT
log |fr| dm < inf, then sumn>=1(1 - |lambdan|) < inf. In
particular, this holds whenever f in Hp(D), p > 0.
 	
3.7.3. Lemma (Blaschke Product). If (lambdan)n>=1 is a
sequence in D satisfying the Blaschke condition sumn>=1(1 -
|lambdan|) < inf, then the infinite product B = prodn>=1
blambdan converges uniformly on compact subsets of D (and even
on compact subsets of C clos{1/lambdan}n>=1). Moreover |B| <=
1 in D, |B| = 1 a.e. on T, and the zeros of B are exactly
(lambdan)n>=1 (counting multiplicities).
 	
3.7.2. Remark. The condition sumn>=1(1 - |lambdan|) < inf is
called the Blaschke condition.
	Hardy spaces	def HardyClass (p : R>=0inf) (f : C -> C) : Prop :=
		  DifferentiableOn C f (Metric.ball (0 : C) 1) /\
		  exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 ->
		  eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p
		  (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C
		
		def BlaschkeCondition (a : N -> C) : Prop :=
		  (forall n : N, a n in Metric.ball (0 : C) 1) /\ Summable (fun n : N => 1
		  - ||a n||)
		
		theorem nikolski_A_3_7_blaschke_zero_sets
		  {p : R>=0inf} {a : N -> C} :
		  (exists f : C -> C, HardyClass p f /\ (exists z, f z != 0) /\
		  forall z in Metric.ball (0 : C) 1, f z = 0 <-> z in range a) <->
		  BlaschkeCondition a := by
		  sorry

5.4.1. Theorem. Let mu be a finite Borel measure on T. The following assertions are equivalent. (1) The family (zn)_{ninZ} is a (symmetric or non-symmetric) basis of L2(mu). (2) The Riesz projection P+ is bounded on L2(mu). (3) sin(Pol+, Pol-) > 0. (4) dmu = |h|2 dm where h in H2 is an outer function such that dist(h/h, Hinf) < 1. (5) dmu = w dm where w = e^{u+v} and u, v are real valued bounded functions	Harmonic analysis	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1}) : C := limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1) noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp>

Natural language statement	Domain	GroundTruth

and ||v||inf < pi/2 (condition (HS)).		def HardyClass (p : R>=0inf) (f : C -> C) : Prop :=
		  DifferentiableOn C f (Metric.ball (0 : C) 1) /\
		  exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 -> eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C
		
		def OuterFunction (p : R>=0inf) (g : C -> C) : Prop :=
		  HardyClass p g /\ exists c : C, ||c|| = 1 /\ forall z in Metric.ball (0 :
		  C) 1, g z = c * Complex.exp ((2 * Real.pi : C)^-1 * integral t in Set.Ioc 0 (2 * Real.pi), ((unitCirclePoint t).1 + z) / ((unitCirclePoint t).1 - z) * Real.log ||boundaryValue g (unitCirclePoint t)||)
		
		noncomputable def trigonometricPolynomial (c : Z -> C) (t : R) : C :=
		  sum' k : Z, c k * Complex.exp (Complex.I * k * t)
		
		noncomputable def weightedL2NormSq (w : {z : C // ||z|| = 1} -> R) (c : Z
		-> C) : R :=
		  integral t in Set.Ioc 0 (2 * Real.pi), ||trigonometricPolynomial c t|| ^ 2 * w (unitCirclePoint t)
		
		def analyticFourierPart (c : Z -> C) (k : Z) : C :=
		  if 0 <= k then c k else 0
		
		noncomputable def circleHilbertTransform (v : {z : C // ||z|| = 1} -> R) (t
		: R) : R :=
		  Complex.re <| sum' k : Z, (if k = 0 then 0 else if 0 < k then -Complex.I else Complex.I) * ((2 * Real.pi : C)^-1 * integral s in Set.Ioc 0 (2 * Real.pi), (v (unitCirclePoint s) : C) * Complex.exp (-Complex.I * k * s)) * Complex.exp (Complex.I * k * t)
		
		theorem nikolski_A_5_4_helson_szego {w : {z : C // ||z|| = 1} -> R} : let basis := exists A B : R, 0 < A /\ A <= B /\ forall c : Z -> C, c.support.Finite -> A * sum' k : Z, ||c k|| ^ 2 <= weightedL2NormSq w c /\ weightedL2NormSq w c <= B * sum' k : Z, ||c k|| ^ 2 let boundedProjection := exists C : R, 0 <= C /\ forall c : Z -> C, c.support.Finite -> weightedL2NormSq w (analyticFourierPart c) <= C ^ 2 * weightedL2NormSq w c let positiveAngle := exists delta : R, 0 < delta /\ forall plus minus : Z -> C, plus.support.Finite -> minus.support.Finite -> (forall k, plus k != 0 -> 0 <= k) -> (forall k, minus k != 0 -> k < 0) -> delta * weightedL2NormSq w plus <= weightedL2NormSq w (fun k => plus k + minus k) let outerDistance := exists h q : C -> C, OuterFunction 2 h /\ HardyClass top q /\ (forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), w (unitCirclePoint t) = ||boundaryValue h (unitCirclePoint t)|| ^ 2) /\ eLpNorm (fun t : R => star (boundaryValue h (unitCirclePoint t)) / boundaryValue h (unitCirclePoint t) - boundaryValue q (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < 1 let helsonSzego := exists u v : {z : C // ||z|| = 1} -> R, eLpNorm (fun t : R => u (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf /\ eLpNorm (fun t : R => v (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < ENNReal.ofReal (Real.pi / 2) /\ forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), w (unitCirclePoint t) = Real.exp (u (unitCirclePoint t) + circleHilbertTransform v t) List.TFAE [basis, boundedProjection, positiveAngle, outerDistance, helsonSzego] := by sorry

Natural language statement	Domain	GroundTruth
1.3.2. Theorem (Z. Nehari, 1957). If H : H2 -> H2- is a bounded Hankel operator, then there exists phi in Linf such that H = H_phi and ‖H_phi‖ = ‖phi‖inf = dist(phi, Hinf).	Operator theory	`noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1})` `: C :=` `  limUnder (nhds[<] (1 : R)) fun r => f (r  *  zeta.1)` `noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} :=` `  <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp>` `def HardyClass (p : R>=0inf) (f : C -> C) : Prop :=` `  DifferentiableOn C f (Metric.ball (0 : C) 1) /\` `    exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 ->` `      eLpNorm (fun t : R => f (r  *  (unitCirclePoint t).1)) p` `        (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C` `noncomputable def circleFourierCoefficient` `    (phi : {z : C // ||z|| = 1} -> C) (k : Z) : C :=` `  (2 * Real.pi : C)^-1 *` `    integral t in Set.Ioc 0 (2 * Real.pi),` `      phi (unitCirclePoint t) * Complex.exp (-Complex.I * k * t)` `def BoundedHankelForm (a : N -> C) : Prop :=` `  exists C : R, 0 <= C /\ forall N : N, forall x y : N -> C,` `    ||Finset.sum (Finset.range N)` `      (fun i => Finset.sum (Finset.range N) (fun j => x i * a (i + j) * y` `      j))|| ^ 2 <=` `      C ^ 2 * Finset.sum (Finset.range N) (fun i => ||x i|| ^ 2) *` `        Finset.sum (Finset.range N) (fun j => ||y j|| ^ 2)` `def HasBoundedHankelSymbol (a : N -> C) (phi : {z : C // ||z|| = 1} -> C) :` `Prop :=` `  AEStronglyMeasurable (fun t : R => phi (unitCirclePoint t))` `      (volume.restrict (Set.Ioc 0 (2 * Real.pi))) /\` `    eLpNorm (fun t : R => phi (unitCirclePoint t)) inf` `      (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf /\` `    forall n : N, circleFourierCoefficient phi (-((n : Z) + 1)) = a n` `noncomputable def hankelFormNorm (a : N -> C) : R>=0inf :=` ` sInf {C : R>=0inf	C < inf /\ forall N : N, forall x y : N -> C,` ` ||Finset.sum (Finset.range N)` ` (fun i => Finset.sum (Finset.range N) (fun j => x i * a (i + j) * y` ` j))|| <=` ` C.toReal * (Finset.sum (Finset.range N) (fun i => ||x i|| ^ 2)) ^ (1` ` / 2 : R) *` ` (Finset.sum (Finset.range N) (fun j => ||y j|| ^ 2)) ^ (1 / 2 : R)}`  `noncomputable def symbolDistanceToHInfinity (phi : {z : C // ||z|| = 1} ->` `C) : R>=0inf :=` ` sInf {r : R>=0inf	exists h : C -> C, HardyClass top h /\` ` eLpNorm (fun t : R =>` ` phi (unitCirclePoint t) - boundaryValue h (unitCirclePoint t)) inf` ` (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= r}`  `theorem nikolski_B_1_3_nehari_theorem` ` {a : N -> C} :` ` BoundedHankelForm a <-> exists phi : {z : C // ||z|| = 1} -> C,` ` HasBoundedHankelSymbol a phi /\` ` eLpNorm (fun t : R => phi (unitCirclePoint t)) inf` ` (volume.restrict (Set.Ioc 0 (2 * Real.pi))) = hankelFormNorm a` ` /\` ` symbolDistanceToHInfinity phi = hankelFormNorm a := by` ` sorry`

2.2.5. Theorem (P. Hartman, 1958). Let f in Linf. Then H_f in Sinf if and only if f in Hinf + C. In other words, a Hankel operator H is compact if and only if H = H_g with some g in C(T).	Operator theory	`noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1})`
		`: C :=`
		`  limUnder (nhds[<] (1 : R)) fun r => f (r  *  zeta.1)`
		
		`noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} :=`
		`  <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp>`
		
		`def HardyClass (p : R>=0inf) (f : C -> C) : Prop :=`
		`  DifferentiableOn C f (Metric.ball (0 : C) 1) /\`
		`    exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 ->`

Natural language statement	Domain	GroundTruth
		eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C noncomputable def circleFourierCoefficient (phi : {z : C // ||z|| = 1} -> C) (k : Z) : C := (2 * Real.pi : C)^-1 * integral t in Set.Ioc 0 (2 * Real.pi), phi (unitCirclePoint t) * Complex.exp (-Complex.I * k * t) def BoundedHankelForm (a : N -> C) : Prop := exists C : R, 0 <= C /\ forall N : N, forall x y : N -> C, ||Finset.sum (Finset.range N) (fun i => Finset.sum (Finset.range N) (fun j => x i * a (i + j) * y j))|| ^ 2 <= C ^ 2 * Finset.sum (Finset.range N) (fun i => ||x i|| ^ 2) * Finset.sum (Finset.range N) (fun j => ||y j|| ^ 2) def HasBoundedHankelSymbol (a : N -> C) (phi : {z : C // ||z|| = 1} -> C) : Prop := AEStronglyMeasurable (fun t : R => phi (unitCirclePoint t)) (volume.restrict (Set.Ioc 0 (2 * Real.pi))) /\ eLpNorm (fun t : R => phi (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf /\ forall n : N, circleFourierCoefficient phi (-((n : Z) + 1)) = a n def CompactHankel (a : N -> C) : Prop := BoundedHankelForm a /\ forall eps : R, 0 < eps -> exists N : N, forall M : N, forall x y : N -> C, ||sum i in Finset.Icc N M, sum j in Finset.Icc N M, x i * a (i + j) * y j|| <= eps * (sum i in Finset.Icc N M, ||x i|| ^ 2) ^ (1 / 2 : R) * (sum j in Finset.Icc N M, ||y j|| ^ 2) ^ (1 / 2 : R) def InHInfinityPlusContinuous (phi : {z : C // ||z|| = 1} -> C) : Prop := exists h : C -> C, exists c : {z : C // ||z|| = 1} -> C, HardyClass top h /\ Continuous c /\ forall zeta : {z : C // ||z|| = 1}, phi zeta = boundaryValue h zeta + c zeta theorem nikolski_B_2_2_hartman_compact_hankel {a : N -> C} : CompactHankel a <-> exists phi : {z : C // ||z|| = 1} -> C, HasBoundedHankelSymbol a phi /\ InHInfinityPlusContinuous phi := by sorry
3.2.4. Corollary (G. Pick, 1916). There exists f in Hinf such that f(lambda_k) = w_k, k = 1, ..., n, and ||f||inf <= 1 if and only if I - WW* >= 0: sum_{i,j=1}^n a_i a_j (1 - w_i w_j)/(1 - lambda_i lambda_j) >= 0, a_i in C. Moreover, the solution f is unique if and only if the matrix I - WW* is degenerated, i.e. d = rank(I - WW*) < n.	Function theory	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1}) : C := limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1) noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp> def HardyClass (p : R>=0inf) (f : C -> C) : Prop := DifferentiableOn C f (Metric.ball (0 : C) 1) /\ exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 -> eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C def SchurFunction (f : C -> C) : Prop := HardyClass top f /\ forall z in Metric.ball (0 : C) 1, ||f z|| <= 1 noncomputable def PickMatrix {n : N} (z w : Fin n -> C) : Matrix (Fin n) (Fin n) C := fun i j => (1 - w i * star (w j)) / (1 - z i * star (z j)) def PositiveSemidefiniteMatrix {n : N} (A : Matrix (Fin n) (Fin n) C) : Prop := forall c : Fin n -> C, 0 <= Complex.re (sum i, sum j, star (c i) * A i j * c j) theorem nikolski_B_3_2_nevanlinna_pick_interpolation

Natural language statement	Domain	GroundTruth
		{n : N} {z w : Fin n -> C} (hz : forall i : Fin n, z i in Metric.ball (0 : C) 1) : let solutions := {f : C -> C | SchurFunction f /\ forall i : Fin n, f (z i) = w i} (solutions.Nonempty <-> PositiveSemidefiniteMatrix (PickMatrix z w)) /\ (solutions.Nonempty -> (solutions.Subsingleton <-> Matrix.det (PickMatrix z w) = 0)) := by sorry
4.3.3. Lemma. Let u in Linf(T) be such that |u| = 1 a.e. on T. The following are equivalent. (1) Tu is invertible. (2) dist(u, Hinf) < 1, dist(u, Hinf) < 1. (3) There exists an outer function h in Hinf such that |u - h|inf < 1. (4) There exist real valued bounded functions a, b and a constant c in R such that u = e^{i(c+a+b)} and |a|inf < pi/2. 3.9.7. Definition. Let f in Hp, p > 0. The function [f] is called the outer part of f, and lambdaBS is called the inner part of f. A function f in Hp which is equal to its outer part (up to a multiplicative constant of modulus 1), f = lambda[f], will simply be called outer.	Operator theory	noncomputable def boundaryValue (f : C -> C) (zeta : {z : C // ||z|| = 1}) : C := limUnder (nhds[<] (1 : R)) fun r => f (r * zeta.1) noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp> def HardyClass (p : R>=0inf) (f : C -> C) : Prop := DifferentiableOn C f (Metric.ball (0 : C) 1) /\ exists C : R>=0inf, C < inf /\ forall r : R, 0 <= r -> r < 1 -> eLpNorm (fun t : R => f (r * (unitCirclePoint t).1)) p (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= C def OuterFunction (p : R>=0inf) (g : C -> C) : Prop := HardyClass p g /\ exists c : C, ||c|| = 1 /\ forall z in Metric.ball (0 : C) 1, g z = c * Complex.exp ((2 * Real.pi : C)^-1 * integral t in Set.Ioc 0 (2 * Real.pi), ((unitCirclePoint t).1 + z) / ((unitCirclePoint t).1 - z) * Real.log ||boundaryValue g (unitCirclePoint t)||) noncomputable def circleHilbertTransform (v : {z : C // ||z|| = 1} -> R) (t : R) : R := Complex.re <| sum' k : Z, (if k = 0 then 0 else if 0 < k then -Complex.I else Complex.I) * ((2 * Real.pi : C)^-1 * integral s in Set.Ioc 0 (2 * Real.pi), (v (unitCirclePoint s) : C) * Complex.exp (-Complex.I * k * s)) * Complex.exp (Complex.I * k * t) noncomputable def circleFourierCoefficient (phi : {z : C // ||z|| = 1} -> C) (k : Z) : C := (2 * Real.pi : C)^-1 * integral t in Set.Ioc 0 (2 * Real.pi), phi (unitCirclePoint t) * Complex.exp (-Complex.I * k * t) noncomputable def symbolDistanceToHInfinity (phi : {z : C // ||z|| = 1} -> C) : R>=0inf := sInf {r : R>=0inf | exists h : C -> C, HardyClass top h /\ eLpNorm (fun t : R => phi (unitCirclePoint t) - boundaryValue h (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= r} def RepresentsToeplitzOperator (phi : {z : C // ||z|| = 1} -> C) (T : l2(N, C) -> l2(N, C)) : Prop := (forall f g, T (f + g) = T f + T g) /\ (forall (c : C) f, T (c * f) = c * T f) /\ Continuous T /\ forall f n, T f n = sum' j : N, circleFourierCoefficient phi ((n : Z) - j) * f j def EssentiallyBoundedCircleSymbol (u : {z : C // ||z|| = 1} -> C) : Prop := AEStronglyMeasurable (fun t : R => u (unitCirclePoint t)) (volume.restrict (Set.Ioc 0 (2 * Real.pi))) /\ eLpNorm (fun t : R => u (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf def IsUnimodularCircleSymbol (u : {z : C // ||z|| = 1} -> C) : Prop := forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), ||u (unitCirclePoint t)|| = 1 theorem nikolski_B_4_3_3_devinatz_widom {u : {z : C // ||z|| = 1} -> C}

Natural language statement	Domain	GroundTruth
		(hu : EssentiallyBoundedCircleSymbol u) (hmod : IsUnimodularCircleSymbol u) : let a := exists T : l2(N, C) -> l2(N, C), RepresentsToeplitzOperator u T /\ Function.Bijective T let b := symbolDistanceToHInfinity u < 1 /\ symbolDistanceToHInfinity (fun zeta => star (u zeta)) < 1 let c := exists h : C -> C, OuterFunction top h /\ eLpNorm (fun t : R => u (unitCirclePoint t) - boundaryValue h (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < 1 let d := exists v w : {z : C // ||z|| = 1} -> R, exists c : R, eLpNorm (fun t : R => v (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < ENNReal.ofReal (Real.pi / 2) /\ eLpNorm (fun t : R => w (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf /\ forall^m t dvolume.restrict (Set.Ioc 0 (2 * Real.pi)), u (unitCirclePoint t) = Complex.exp (Complex.I * (c + v (unitCirclePoint t) + circleHilbertTransform w t)) List.TFAE [a, b, c, d] := by sorry
7.2.1. Theorem (V. Adamyan, D. Arov, and M. Krein, 1971). Let Hphi be a Hankel operator and let Rn be the set of rational functions tending to 0 at infinity and having all poles in D such that their total multiplicity is less than or equal to n. Then sn(Hphi) = min{||Hphi - Hpsi|| : rank Hpsi <= n} = distlinf(phi, Rn + Hinf) = min{||H_{Bphi}|| : B is a Blaschke product of degree <= n}, where the degree deg Theta of an inner function Theta is equal to n if Theta is a finite Blaschke product with n zeros (counting multiplicities) and inf otherwise.	Operator theory	noncomputable def unitCirclePoint (t : R) : {z : C // ||z|| = 1} := <Complex.exp (Complex.I * t), by rw [Complex.norm_exp]; simp> noncomputable def circleFourierCoefficient (phi : {z : C // ||z|| = 1} -> C) (k : Z) : C := (2 * Real.pi : C)^-1 * integral t in Set.Ioc 0 (2 * Real.pi), phi (unitCirclePoint t) * Complex.exp (-Complex.I * k * t) def BoundedHankelForm (a : N -> C) : Prop := exists C : R, 0 <= C /\ forall N : N, forall x y : N -> C, ||Finset.sum (Finset.range N) (fun i => Finset.sum (Finset.range N) (fun j => x i * a (i + j) * y j))|| ^ 2 <= C ^ 2 * Finset.sum (Finset.range N) (fun i => ||x i|| ^ 2) * Finset.sum (Finset.range N) (fun j => ||y j|| ^ 2) def HasBoundedHankelSymbol (a : N -> C) (phi : {z : C // ||z|| = 1} -> C) : Prop := AEStronglyMeasurable (fun t : R => phi (unitCirclePoint t)) (volume.restrict (Set.Ioc 0 (2 * Real.pi))) /\ eLpNorm (fun t : R => phi (unitCirclePoint t)) inf (volume.restrict (Set.Ioc 0 (2 * Real.pi))) < inf /\ forall n : N, circleFourierCoefficient phi (-((n : Z) + 1)) = a n theorem nikolski_B_7_2_1_adamyan_arov_krein {a : N -> C} {phi : {z : C // ||z|| = 1} -> C} {n : N} : HasBoundedHankelSymbol a phi -> forall r : R, (0 <= r /\ r = sInf {C : R | 0 <= C /\ exists b : N -> C, BoundedHankelForm b /\ exists correction : Fin n -> N -> C, forall N : N, forall x y : N -> C, ||sum i in Finset.range N, sum j in Finset.range N, x i * (a (i + j) - b (i + j) - sum q, correction q i * correction q j) * y j|| <= C * (sum i in Finset.range N, ||x i|| ^ 2) ^ (1 / 2 : R) * (sum j in Finset.range N, ||y j|| ^ 2) ^ (1 / 2 : R)}) <-> (0 <= r /\ r = sInf {C : R | 0 <= C /\ exists psi : {z : C // ||z|| = 1} -> C, (exists numerator denominator : Polynomial C, denominator.natDegree <= n /\ (forall zeta : {z : C // ||z|| = 1}, denominator.eval zeta.1 != 0) /\ forall zeta : {z : C // ||z|| = 1}, psi zeta = numerator.eval zeta.1 / denominator.eval zeta.1) /\ eLpNorm (fun t : R => phi (unitCirclePoint t) - psi (unitCirclePoint t)) inf

Natural language statement	Domain	GroundTruth
		(volume.restrict (Set.Ioc 0 (2 * Real.pi))) <= ENNReal.ofReal C}) := by sorry